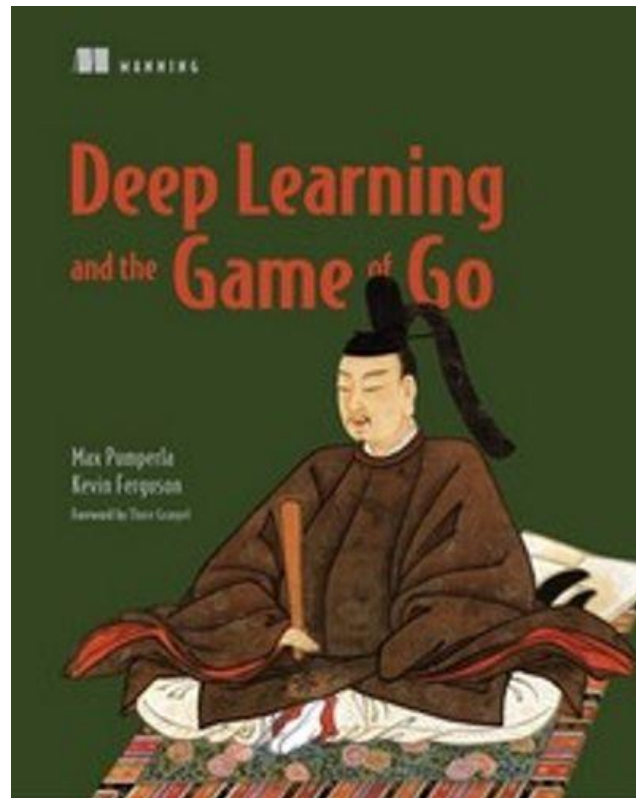


체스 Agent 제작 1주차

체스 보드 및 무작위 Agent 제작

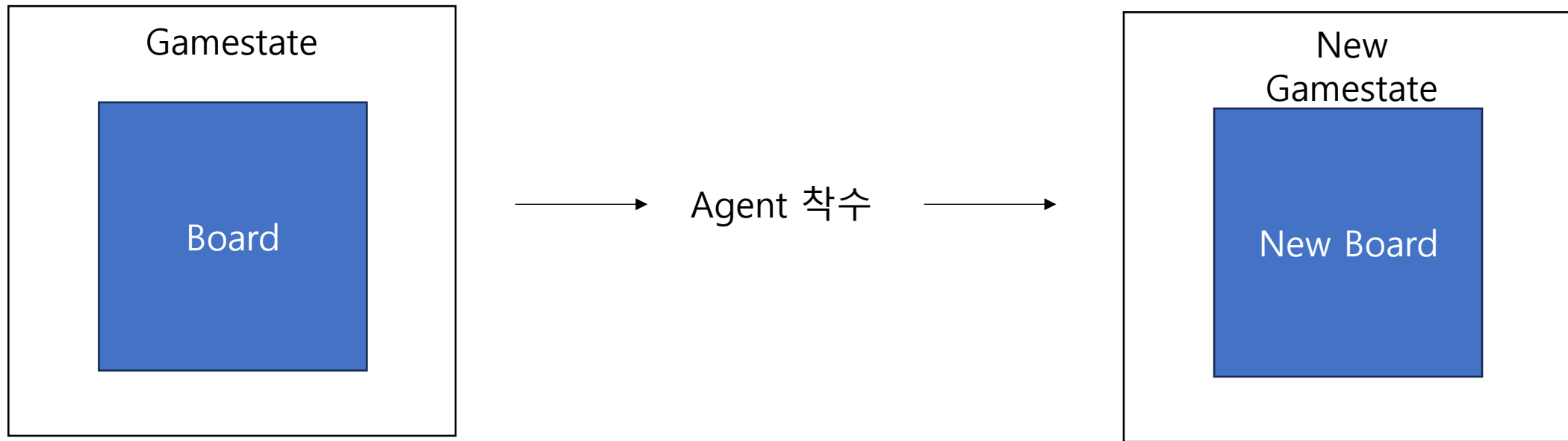
1. 참고 서적

- Deep Learning and the Game of Go 책의 구조를 기반으로 프로그램을 작성함(중앙도서관에서 대여 가능)
- 바둑 프로그램의 구조를 기반으로 체스 프로그램으로 변형
- https://github.com/maxpumperla/deep_learning_and_the_game_of_go 참조



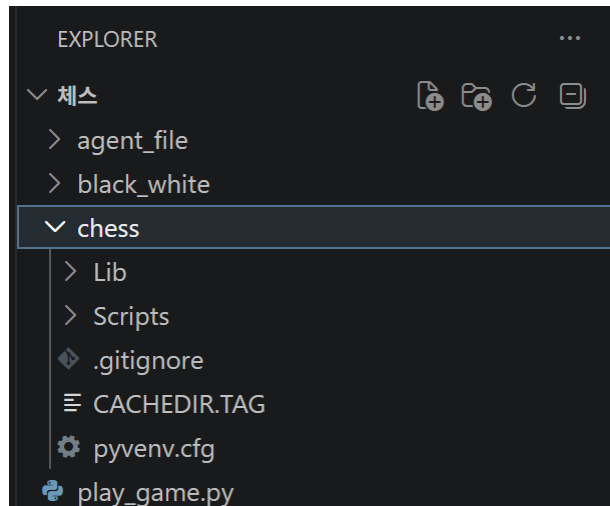
2. Agent 구조

- 3가지 구성 요소: Board, Gamestate, Agent
 - Board: 게임판 위에 올라가 있는 기물의 위치를 저장
 - Gamestate: Board의 정보를 기반으로 게임의 진행을 주관하고 승패를 판정
 - Agent: 게임을 두는 컴퓨터 플레이어, Gamestate를 받고 적절한 수를 둬

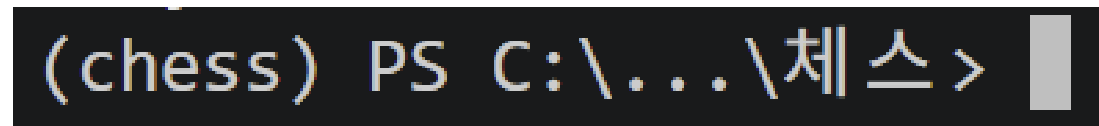


3. 사전 준비 - 가상환경 설정

- 체스 프로그램에 필요한 패키지를 설치하기 위한 가상환경 설정
 - vscode에서 프로젝트를 진행할 폴더를 열고, terminal에 “python -m venv 가상환경이름” 입력
 - 성공 시 프로젝트 폴더 내에 가상환경 관련 폴더 생성
 - 프로젝트 폴더를 열 때 항상 가상환경이 실행될 수 있도록 terminal에 “Set-ExecutionPolicy - ExecutionPolicy RemoteSigned -Scope CurrentUser” 입력
 - 성공 시 <그림 1>과 같은 파일 구조, <그림 2>와 같이 프롬프트 앞에 (가상환경이름)이 표시됨(chess는 임의의 가상환경 이름임)



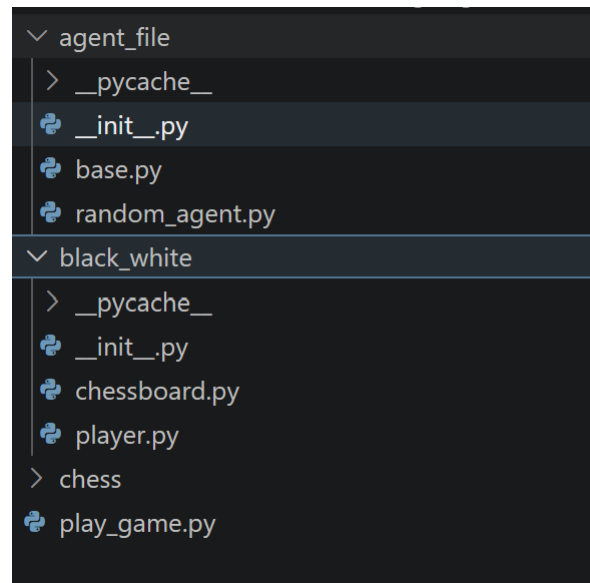
<그림 1>



<그림 2>

4. 파일 구조 설정

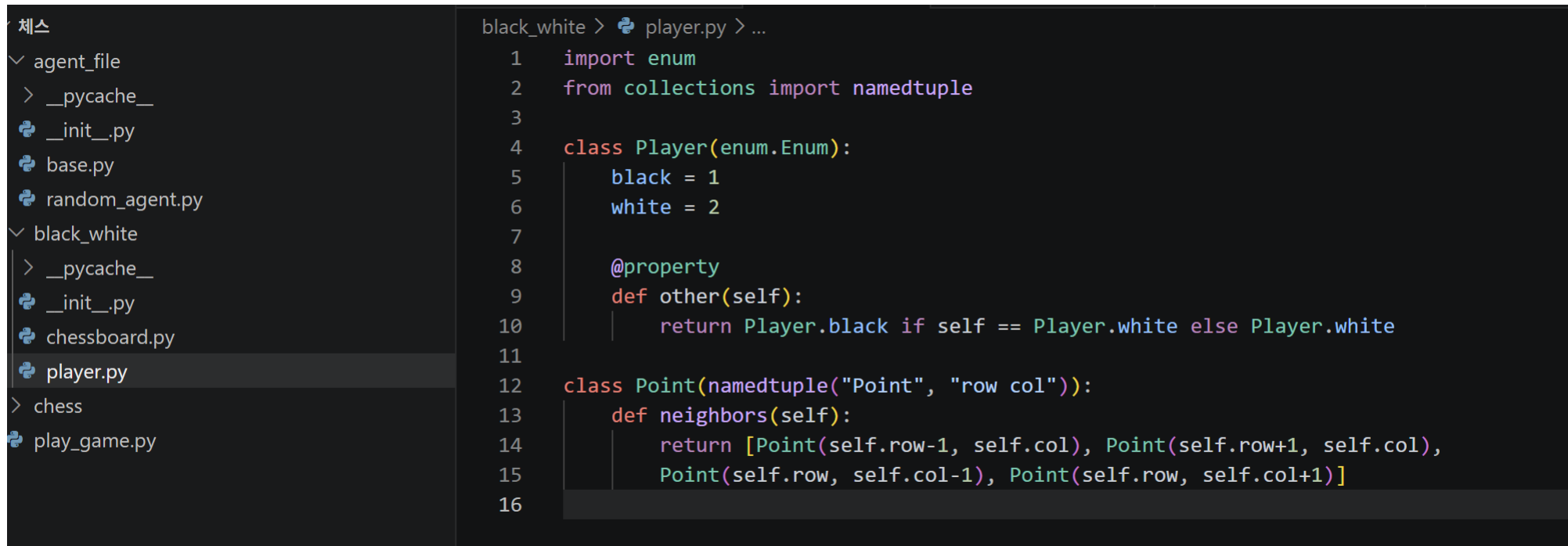
- 체스 프로그램에 필요한 패키지를 설치하기 위한 가상환경 설정
 - Board, Gamestate 등 게임 상태를 관리할 파일들은 black_white 폴더에 저장
 - Agent 관련 내용의 경우 agent_file 폴더에 저장
 - 각 폴더 내에 __init__.py라는 파일을 생성(__init__.py는 폴더가 python 패키지임을 알려주는 용도이므로, 파일 내에 아무 내용도 적지 않아도 됨)
 - __pycache__는 자동으로 생성되는 파일이고, 나머지 .py 파일은 추후 작성할 것이므로 참고만 해도 됨



<그림 3>

5. 게임 환경 제작 - player.py

- Board, Gamestate에 필요한 요소들을 준비
 - black_white 폴더 내에 player.py 파일을 생성함
 - 다음 수에 둘 차례를 확인하는 Player class와, Board에서의 위치를 정의하는 Point class



```
체스
├── agent_file
│   ├── __pycache__
│   ├── __init__.py
│   ├── base.py
│   └── random_agent.py
├── black_white
│   ├── __pycache__
│   ├── __init__.py
│   ├── chessboard.py
│   └── player.py
├── chess
└── play_game.py

black_white > player.py > ...
1  import enum
2  from collections import namedtuple
3
4  class Player(enum.Enum):
5      black = 1
6      white = 2
7
8      @property
9      def other(self):
10         return Player.black if self == Player.white else Player.white
11
12  class Point(namedtuple("Point", "row col")):
13      def neighbors(self):
14         return [Point(self.row-1, self.col), Point(self.row+1, self.col),
15                 Point(self.row, self.col-1), Point(self.row, self.col+1)]
16
```

<그림 4>

5. 게임 환경 제작 - player.py

- Player class: Board에서 다음에 둘 플레이어가 흑색인지, 백색인지를 결정
- enum: 정해진 선택지들을 이름으로 관리하기 위해 존재함(ex. player = Player.black 등으로 명확하게 지정 가능) + black/white 외에 Player class가 다른 값을 가지지 못하도록 함(Player.blue 등 다른 값 지정 불가)
- Other 함수: next_player = previous_player.other와 같이 다음 플레이어로 전환하기 위한 함수 (@property는 player.other() 대신 player.other 등으로 쉽게 호출할 수 있게 함)

```
import enum
from collections import namedtuple

class Player(enum.Enum):
    black = 1
    white = 2

    @property
    def other(self):
        return Player.black if self == Player.white else Player.white

class Point(namedtuple("Point", "row col")):
    def neighbors(self):
        return [Point(self.row-1, self.col), Point(self.row+1, self.col),
                Point(self.row, self.col-1), Point(self.row, self.col+1)]
```

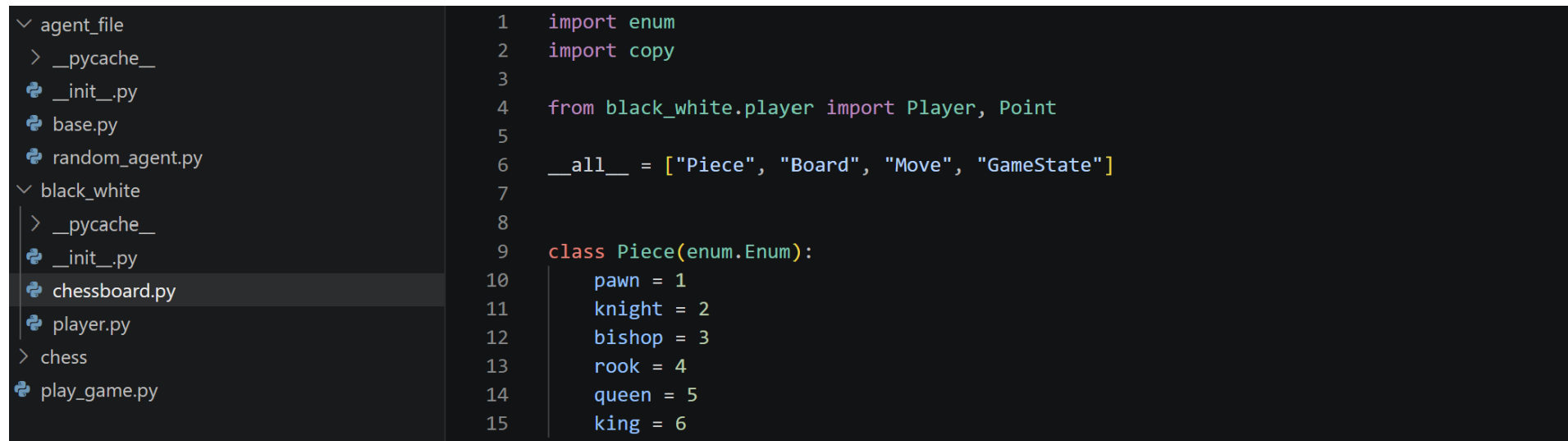
5. 게임 환경 제작 - player.py

- Point class: Board 위의 좌표를 다루는 class
 - Point(namedtuple("Point", "row col")):
 - 1) namedtuple("Point", "row col"): 이름이 Point이고, 각 원소를 [0], [1] 대신 .row, .col로 호출할 수 있음(ex. P = Point(1,4)라 하면 P.row -> 1 출력, P.col -> 4 출력)
 - 2) Point class에 상속: Point 자체를 class로 만들어, 추가 함수를 사용할 수 있도록 함
 - neighbors: Point 객체를 받아, 주변 4곳의 좌표를 돌려줌

```
class Point(namedtuple("Point", "row col")):  
    def neighbors(self):  
        return [Point(self.row-1, self.col), Point(self.row+1, self.col),  
                Point(self.row, self.col-1), Point(self.row, self.col+1)]
```

5. 게임 환경 제작 - chessboard.py

- Board, Gamestate class를 마련하는 파일
 - black_white 폴더 내에 chessboard.py 파일을 넣음
 - black_white의 player.py에서 Player와 Point를 불러옴
 - 주요 class: Piece, Board, Move, Gamestate

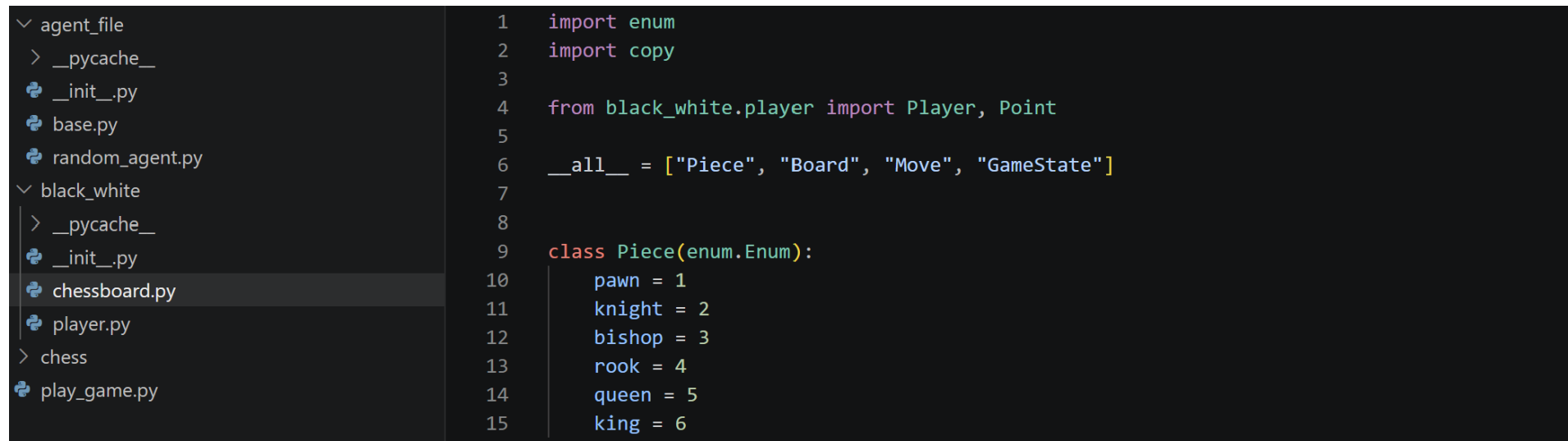


```
1  import enum
2  import copy
3
4  from black_white.player import Player, Point
5
6  __all__ = ["Piece", "Board", "Move", "GameState"]
7
8
9  class Piece(enum.Enum):
10     pawn = 1
11     knight = 2
12     bishop = 3
13     rook = 4
14     queen = 5
15     king = 6
```

<그림 5>

5. 게임 환경 제작 - chessboard.py

- Board, Gamestate class를 마련하는 파일
 - black_white 폴더 내에 chessboard.py 파일을 넣음
 - black_white의 player.py에서 Player와 Point를 불러옴
 - 주요 class: Piece, Move, Board, Gamestate



```
1  import enum
2  import copy
3
4  from black_white.player import Player, Point
5
6  __all__ = ["Piece", "Board", "Move", "GameState"]
7
8
9  class Piece(enum.Enum):
10     pawn = 1
11     knight = 2
12     bishop = 3
13     rook = 4
14     queen = 5
15     king = 6
```

<그림 5>

5. 게임 환경 제작 - chessboard.py

- Piece, Move class
- Piece: 체스판의 주요 기물 종류
- Move: 출발 위치, 도착 위치, 승진 시 바뀔 말의 종류(폰의 경우) 저장
 - `__init__`: class 호출 시 자동 시행, 주요 변수 할당
 - `__eq__`: 서로 다른 Move objec를 비교하는 데 사용
 - `__hash__`: Move 값을 set이나 dictionary의 key로 사용하기 위해 필요함
 - `__repr__`: Move 객체를 print했을 때 나오는 결과값 정의

```
class Piece(enum.Enum):
    pawn = 1
    knight = 2
    bishop = 3
    rook = 4
    queen = 5
    king = 6

class Move:
    def __init__(self, point_from, point_to, promotion=None):
        self.point_from = point_from
        self.point_to = point_to
        self.promotion = promotion

    def __eq__(self, other):
        return (
            isinstance(other, Move)
            and self.point_from == other.point_from
            and self.point_to == other.point_to
            and self.promotion == other.promotion
        )

    def __hash__(self):
        return hash((self.point_from, self.point_to, self.promotion))

    def __repr__(self):
        return f"Move({self.point_from} -> {self.point_to})"
```

5. 게임 환경 제작 - chessboard.py

- Board class
 - `__init__`: 보드판 정보 저장용 dictionary 인 `grid` 정의 / `_setup_initial_position()` 으로 체스판 초기 조건 설정
 - `_setup_initial_position`: `grid`에 각 기물 위치 배정

```
class Board:
    def __init__(self):
        self.grid = {}
        self._setup_initial_position()

    def _setup_initial_position(self):
        self.grid.clear()

        # White pieces
        self.grid[Point(0, 0)] = (Player.white, Piece.rook)
        self.grid[Point(0, 1)] = (Player.white, Piece.knight)
        self.grid[Point(0, 2)] = (Player.white, Piece.bishop)
        self.grid[Point(0, 3)] = (Player.white, Piece.queen)
        self.grid[Point(0, 4)] = (Player.white, Piece.king)
        self.grid[Point(0, 5)] = (Player.white, Piece.bishop)
        self.grid[Point(0, 6)] = (Player.white, Piece.knight)
        self.grid[Point(0, 7)] = (Player.white, Piece.rook)

        for col in range(8):
            self.grid[Point(1, col)] = (Player.white, Piece.pawn)

        # Black pieces
        self.grid[Point(7, 0)] = (Player.black, Piece.rook)
        self.grid[Point(7, 1)] = (Player.black, Piece.knight)
        self.grid[Point(7, 2)] = (Player.black, Piece.bishop)
        self.grid[Point(7, 3)] = (Player.black, Piece.queen)
        self.grid[Point(7, 4)] = (Player.black, Piece.king)
        self.grid[Point(7, 5)] = (Player.black, Piece.bishop)
        self.grid[Point(7, 6)] = (Player.black, Piece.knight)
        self.grid[Point(7, 7)] = (Player.black, Piece.rook)

        for col in range(8):
            self.grid[Point(6, col)] = (Player.black, Piece.pawn)
```

5. 게임 환경 제작 - chessboard.py

- Board class
 - is_on_board: 보드판 내에 위치한 점인지 판별
 - get: grid에서 특정 point에 놓인 기물 탐색(기물이 없으면 None 변환)
 - place: 특정 point에 기물을 배치
 - remove: 특정 point의 기물을 제거
 - is_empty: 특정 point가 비어있는지 판별
 - is_occupied_by: 특정 point에 기물이 위치하고, 입력된 player와 동일한지 판별

```
def is_on_board(self, point):  
    return 0 <= point.row < 8 and 0 <= point.col < 8  
  
def get(self, point):  
    return self.grid.get(point)  
  
def place(self, point, player, piece):  
    if not self.is_on_board(point):  
        raise ValueError("point is outside the board")  
  
    self.grid[point] = (player, piece)  
  
def remove(self, point):  
    if point in self.grid:  
        del self.grid[point]  
  
def is_empty(self, point):  
    return self.get(point) is None  
  
def is_occupied_by(self, point, player):  
    item = self.get(point)  
    return item is not None and item[0] == player
```

5. 게임 환경 제작 - chessboard.py

- Board class
 - find_king: 특정 player의 king 위치 반환
 - place_stone: 원래 board에서 착수 후 변한 새로운 board를 생성하는 함수, move 객체를 입력값으로 받음
- 1) new_board를 deepcopy로 복사:
new_board의 작업이 기존 board 객체를 바꾸는 것을 방지하기 위함
- 2) 옮길 기물의 player와 기물 종류를 player, piece로 저장
- 3) 옮긴 말의 기존 위치 정보 삭제
- 4) Promotion이 있을 경우 piece 종류 변경
- 5) 새로운 위치에 기물 배치
- 6) New_board 객체 반환

```
def find_king(self, player):
    for point, item in self.grid.items():
        item_player, piece = item
        if item_player == player and piece == Piece.king:
            return point

    return None

def place_stone(self, move):
    new_board = copy.deepcopy(self)

    player, piece = new_board.grid[move.point_from]

    del new_board.grid[move.point_from]

    if move.promotion is not None:
        piece = move.promotion

    new_board.grid[move.point_to] = (player, piece)

    return new_board
```

5. 게임 환경 제작 - chessboard.py

- GameState class: Board 객체를 받아 다음 수가 적법한 수인지, 기물의 움직임은 어떠한지, 게임이 종료된 상태인지 등 게임 규칙에 관련된 처리를 담당하는 class
 - __init__: 현재 게임 상태에 해당하는 Board와 이번 턴에 둘 player를 self.board와 self.next_player로 가짐

```
class GameState:
    def __init__(self, board=None, next_player=Player.white):
        self.board = board if board is not None else Board()
        self.next_player = next_player
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - legal_moves: 다음에 둘 player의 기물이 이동할 수 있는 모든 위치를 기록
 - 1) _moves_for_piece: 기물 1개의 이동 가능 위치 반환
 - 2) _move_leaves_player_in_check: 체크메이트에 노출되는 수를 제외
 - apply_moves: move에 있는 수가 적법한 수인지 검사한 후, 해당 수가 적용된 GameState 객체를 반환함

```
def legal_moves(self):
    moves = []

    for point, item in self.board.grid.items():
        player, piece = item

        if player == self.next_player:
            candidate_moves = self._moves_for_piece(point, player, piece)

            for move in candidate_moves:
                if not self._move_leaves_player_in_check(move, player):
                    moves.append(move)

    return moves

def apply_move(self, move):
    if move not in self.legal_moves():
        raise ValueError("illegal move")

    next_board = self.board.place_stone(move)

    return GameState(
        board=next_board,
        next_player=self.next_player.other,
    )
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - is_over: 게임 종료 판단, 더 이상 둘 수 있는 수가 없을 때 True 반환
 - Winner: 게임이 끝났을 때 체크메이트 상태를 얻은 player를 승리자로 반환
 - is_checkmate, is_stalemate: 게임 종료 상태일 때의 상태를 판별함

```
def is_over(self):  
    return len(self.legal_moves()) == 0  
  
def winner(self):  
    if not self.is_over():  
        return None  
  
    if self.is_in_check(self.next_player):  
        return self.next_player.other  
  
    return None  
  
def is_checkmate(self):  
    return self.is_over() and self.is_in_check(self.next_player)  
  
def is_stalemate(self):  
    return self.is_over() and not self.is_in_check(self.next_player)
```

5. 게임 환경 제작 - chessboard.py

- GameState class

- is_in_check: 현재 player의 king의 위치를 파악하고, 다른 player의 기물이 king 위치에 도달할 수 있는지 검사

1) _pseudo_moves_for_piece: 특정 piece가 이동할 수 있는 좌표를 기반으로, move object들을 반환함

```
def is_in_check(self, player):
    king_point = self.board.find_king(player)

    if king_point is None:
        return True

    opponent = player.other

    for point, item in self.board.grid.items():
        item_player, piece = item

        if item_player == opponent:
            opponent_moves = self._pseudo_moves_for_piece(point, item_player, piece)

            for move in opponent_moves:
                if move.point_to == king_point:
                    return True

    return False
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - `_move_leaves_player_in_check`: 특정 move가 체크메이트 상태를 벗어나게 할 수 있는지 검사
 - `_moves_for_piece`:
`_pseudo_moves_for_piece`와 동일하지만, 이후 캐슬링이나 암파상 등을 추가적으로 구현할 수 있도록 마련한 함수(여담: 저도 체스 규칙을 공부하고 GPT를 이용해 코드를 제작하고 있어, 조 내에서 체스를 잘 아시는 분들의 도움이 필요합니다...)

```
def _move_leaves_player_in_check(self, move, player):  
    next_board = self.board.place_stone(move)  
    next_state = GameState(next_board, player.other)  
  
    return next_state.is_in_check(player)  
  
def _moves_for_piece(self, point, player, piece):  
    return self._pseudo_moves_for_piece(point, player, piece)
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - `_move_leaves_player_in_check`: 특정 move가 체크메이트 상태를 벗어나게 할 수 있는지 검사
 - `_moves_for_piece`:
`_pseudo_moves_for_piece`와 동일하지만, 이후 캐슬링이나 암파상 등을 추가적으로 구현할 수 있도록 마련한 함수(여담: 저도 체스 규칙을 공부하고 GPT를 이용해 코드를 제작하고 있어, 조 내에서 체스를 잘 아시는 분들의 도움이 필요합니다...)

```
def _move_leaves_player_in_check(self, move, player):  
    next_board = self.board.place_stone(move)  
    next_state = GameState(next_board, player.other)  
  
    return next_state.is_in_check(player)  
  
def _moves_for_piece(self, point, player, piece):  
    return self._pseudo_moves_for_piece(point, player, piece)
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - `_move_leaves_player_in_check`: 특정 move가 체크메이트 상태를 벗어나게 할 수 있는지 검사
 - `_moves_for_piece`:
`_pseudo_moves_for_piece`와 동일하지만, 이후 캐슬링이나 암파상 등을 추가적으로 구현할 수 있도록 마련한 함수(여담: 저도 체스 규칙을 공부하고 GPT를 이용해 코드를 제작하고 있어, 조 내에서 체스를 잘 아시는 분들의 도움이 필요합니다...)

```
def _move_leaves_player_in_check(self, move, player):
    next_board = self.board.place_stone(move)
    next_state = GameState(next_board, player.other)

    return next_state.is_in_check(player)

def _moves_for_piece(self, point, player, piece):
    return self._pseudo_moves_for_piece(point, player, piece)
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - pseudo_moves_for_piece: 특정 piece가 취할 수 있는 모든 move object들을 모음
- 1) _sliding_moves: 가로/세로/대각선 움직임을 취하는 기물의 움직임 담당, direction의 기물기는 가능한 움직임 방향
 - 2) _pawn_moves: 폰의 움직임 담당
 - 3) _knight_moves: 나이트의 움직임 담당
 - 4) _king_moves: 킹의 움직임 담당

```
def _pseudo_moves_for_piece(self, point, player, piece):
    if piece == Piece.pawn:
        return self._pawn_moves(point, player)

    if piece == Piece.knight:
        return self._knight_moves(point, player)

    if piece == Piece.bishop:
        return self._sliding_moves(
            point,
            player,
            directions=[(1, 1), (1, -1), (-1, 1), (-1, -1)],
        )

    if piece == Piece.rook:
        return self._sliding_moves(
            point,
            player,
            directions=[(1, 0), (-1, 0), (0, 1), (0, -1)],
        )

    if piece == Piece.queen:
        return self._sliding_moves(
            point,
            player,
            directions=[
                (1, 0), (-1, 0), (0, 1), (0, -1),
                (1, 1), (1, -1), (-1, 1), (-1, -1),
            ],
        )

    if piece == Piece.king:
        return self._king_moves(point, player)

    return []
```

5. 게임 환경 제작 - chessboard.py

- Gamestate class

- `_pawn_moves`: pawn이 취할 수 있는 모든 move object를 반환

- 1) `direction`, `start_row`, `last_row`: player의 색깔에 따른 폰의 전진 방향, 시작 및 최종 row 위치
- 2) `One_step`: 폰의 바로 앞 위치 좌표
- 3) `One_step` 위치가 비어있을 경우 가능한 move 탐색 시작
- 4) 폰이 1칸 전진한 위치가 끝 row면 promotion에 queen 할당 후 move object 할당, 그렇지 않은 경우 일반 move object 할당
- 5) 폰이 시작 row에 있을 경우 앞으로 2칸 전진 가능
- 6) For `dc in [-1, 1]` 이하 코드: 앞쪽 대각선 위치에 잡을 수 있는 기물이 있으면 이동 가능한 범위에 포함함

```
def _pawn_moves(self, point, player):
    moves = []

    direction = 1 if player == Player.white else -1
    start_row = 1 if player == Player.white else 6
    last_row = 7 if player == Player.white else 0

    one_step = Point(point.row + direction, point.col)

    if self.board.is_on_board(one_step) and self.board.is_empty(one_step):
        promotion = Piece.queen if one_step.row == last_row else None
        moves.append(Move(point, one_step, promotion))

    two_step = Point(point.row + 2 * direction, point.col)

    if (
        point.row == start_row
        and self.board.is_on_board(two_step)
        and self.board.is_empty(two_step)
    ):
        moves.append(Move(point, two_step))

    for dc in [-1, 1]:
        target = Point(point.row + direction, point.col + dc)

        if not self.board.is_on_board(target):
            continue

        item = self.board.get(target)

        if item is not None:
            target_player, _ = item

            if target_player != player:
                promotion = Piece.queen if target.row == last_row else None
                moves.append(Move(point, target, promotion))

    return moves
```

5. 게임 환경 제작 - chessboard.py

- GameState class
 - `_knight_moves`: knight이 취할 수 있는 move 들을 반환함
 - `_king_moves`: king이 취할 수 있는 모든 move 들을 취함

참고: `_king_moves`에서 가능한 move 중 king이 죽는 위치가 끼어 들어가더라도 `legal_moves`에서 체크메이트 위치를 걸러주므로 문제는 발생하지 않음

```
def _knight_moves(self, point, player):
    moves = []
    candidates = [
        Point(point.row + 2, point.col + 1),
        Point(point.row + 2, point.col - 1),
        Point(point.row - 2, point.col + 1),
        Point(point.row - 2, point.col - 1),
        Point(point.row + 1, point.col + 2),
        Point(point.row + 1, point.col - 2),
        Point(point.row - 1, point.col + 2),
        Point(point.row - 1, point.col - 2),
    ]

    for target in candidates:
        if not self.board.is_on_board(target):
            continue

        if not self.board.is_occupied_by(target, player):
            moves.append(Move(point, target))
    return moves

def _king_moves(self, point, player):
    moves = []

    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            if dr == 0 and dc == 0:
                continue

            target = Point(point.row + dr, point.col + dc)

            if not self.board.is_on_board(target):
                continue

            if not self.board.is_occupied_by(target, player):
                moves.append(Move(point, target))
    return moves
```

5. 게임 환경 제작 - chessboard.py

- GameState class

- `_sliding_moves`: 비숍, 룯, 퀸 등 직선적으로 움직이는 말들의 가능한 move들을 반환하는 함수

- 1) 특정 방향(directions)으로 모든 위치들을 검사하면서, 다른 기물에 막히기 전까지 갈 수 있는 모든 위치로 향하는 move object를 반환

- 2) 막는 기물이 상대편 기물일 경우 이를 잡을 수 있으므로 해당 위치도 포함함

- `__repr__`: gamestate 객체를 print할 때 board값을 출력하도록 함

```
def _sliding_moves(self, point, player, directions):
    moves = []

    for dr, dc in directions:
        row = point.row + dr
        col = point.col + dc

        while True:
            target = Point(row, col)

            if not self.board.is_on_board(target):
                break

            item = self.board.get(target)

            if item is None:
                moves.append(Move(point, target))
            else:
                target_player, _ = item

                if target_player != player:
                    moves.append(Move(point, target))

                break

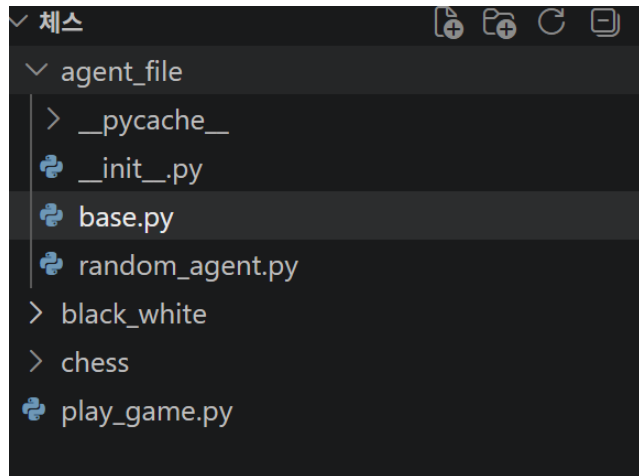
            row += dr
            col += dc

        return moves

def __repr__(self):
    return repr(self.board)
```

6. 무작위 Agent 제작

- 게임판 관련 class들을 모두 설정하였으므로, 게임을 플레이하는 Agent 제작
 - 앞서 생성한 Agent_file 폴더 내에 __init__.py, base.py, random_agent.py를 생성
 - __init__.py 내에는 아무 내용도 입력하지 않아도 됨
 - Base.py 내에 class Agent와 내부에 select_move 함수를 작성 -> 해당 내용은 이후에 작성될 모든 Agent 관련 class들이 상속할 것



<그림 6>
Agent_file 내 구조

```
1  __all__=["Agent"]
2
3  class Agent:
4      def __init__(self):
5          pass
6
7      def select_move(self,game_state):
8          raise NotImplementedError()
```

<그림 7>
Base.py 내용

6. 무작위 Agent 제작

- 무작위 수를 두는 Randombot 제작 (random_agent.py 내부에 작성)
- Randombot class
 - `__init__`: 부모 class인 Agent의 내용을 상속받음
 - `Select_move`: `game_state`에서 가능한 move들 중 아무 것이나 무작위로 선택함

```
import random

from black_white.chessboard import *
from agent_file.base import Agent

__all__ = ["Randombot"]

class Randombot(Agent):
    def __init__(self):
        super().__init__()

    def select_move(self, game_state):
        moves = game_state.legal_moves()

        if not moves:
            return None

        return random.choice(moves)
```

6. 무작위 Agent 제작

- 실제 game을 플레이하는 데 사용되는 코드 제작
 - 모든 폴더 바깥에 play_game.py 파일 생성
 - Play_game.py 내부 내용:
 - print_game_result: 게임 종료 후 게임 결과를 출력
 - Main 함수
 - 1) 백, 흑 player에 Randombot 객체 할당
 - 2) 최대 300회 move 설정
 - 3) Game이 끝나기 전까지, 다음 순서에 해당하는 player를 호출(이 경우 Randombot) 후 gamestate 객체(game)를 select_move 함수에 입력하여 다음 move를 얻음
 - 4) 해당 move를 적용한 새로운 game 객체를 만들고 출력
 - 5) 게임이 끝날 때까지 해당 과정 반복
 - If __name__ == "__main__": terminal에 play_game.py 함수를 실행했을 때 main() 함수가 실행되게 함
- 참고) “__main__”은 main() 함수가 아니라 terminal에 해당 파일이 직접 실행되었는지를 확인하는 용도임

```
from black_white.player import Player
from black_white.chessboard import GameState
from agent_file.random_agent import Randombot
def print_game_result(game):
    if game.is_checkmate():
        print("Checkmate!")
        print("Winner:", game.winner())
    elif game.is_stalemate():
        print("Stalemate!")
    else:
        print("Game stopped.")

def main():
    game = GameState()

    agents = {
        Player.white: Randombot(),
        Player.black: Randombot(),
    }

    max_moves = 300
    move_count = 0

    print("Initial board")
    print(game)
    print()
    while not game.is_over() and move_count < max_moves:
        current_player = game.next_player
        agent = agents[current_player]

        move = agent.select_move(game)

        if move is None:
            break
        move_count += 1
        print(f"Move {move_count}: {current_player.name} plays {move}")
        game = game.apply_move(move)
        print(game)
        print()
        print_game_result(game)
if __name__ == "__main__":
    main()
```

6. 무작위 Agent 제작

- 실행 결과: 게임 진행 양상이 terminal에 출력될 것임.

```
Move 218: black plays Move(Point(row=1, col=1) -> Point(row=1, col=5))
7 . . . . . . .
6 . . . . . . .
5 . n . . . . . Q
4 p . . . . K . .
3 . . . . . . P
2 . . k . . B P .
1 . . . . . q . .
0 . . . . . . .

  0 1 2 3 4 5 6 7

Move 219: white plays Move(Point(row=5, col=7) -> Point(row=4, col=7))
7 . . . . . . .
6 . . . . . . .
5 . n . . . . .
4 p . . . . K . Q
3 . . . . . . P
2 . . k . . B P .
1 . . . . . q . .
0 . . . . . . .

  0 1 2 3 4 5 6 7

Move 220: black plays Move(Point(row=5, col=1) -> Point(row=7, col=2))
7 . . n . . . .
6 . . . . . . .
5 . . . . . . .
4 p . . . . K . Q
3 . . . . . . P
2 . . k . . B P .
1 . . . . . q . .
0 . . . . . . .

  0 1 2 3 4 5 6 7
```

<그림 8>
게임 진행 출력

7. 다음 시간 내용 및 개선 과제

- 이번 주차를 통해 체스판과 체스 규칙을 설정하였으며, 기초적인 Agent를 제작함
- 다음 주차에는 바로 인공지능 Agent 제작을 들어가기 전, 트리 탐색을 기반으로 수를 두는 Agent를 제작할 예정임. 이렇게 제작된 모델을 인공지능 Agent의 훈련에 사용하고자 함.
- 개선점: 인간과도 대국할 수 있는 시스템 제작, GUI 환경 제작
- 각 조에서 제공된 파일을 기반으로 자유롭게 개선해 나가 주시면 될 것 같습니다..!

감사합니다!